

SDP



Copilot Configuration Primitives

Instructions · Skills · Custom Prompts · Agents

Encode your codebase once. Every Copilot interaction inherits it.

"How can I make GitHub Copilot actually understand my codebase?"

Generic responses ignore your team's conventions. The fix is not better prompting — it's configuration.

Four primitives. Committed to the repo. Every interaction inherits your team's knowledge.

Individual contributors

Developers who paste the same context into every chat session and want Copilot to just know their project conventions.

Repeated context today

The average developer re-explains the same project conventions to Copilot in every single chat session — minutes wasted, every day.

Team leads

Engineering leads who want consistent, team-wide Copilot behavior — not per-developer variation in response quality.

5-minute setup

One copilot-instructions.md file transforms Copilot from generic assistant to team-aware partner — visible improvement on the first...

Platform engineers

Infrastructure engineers automating workflows who need Copilot to understand domain-specific tools and constraints.

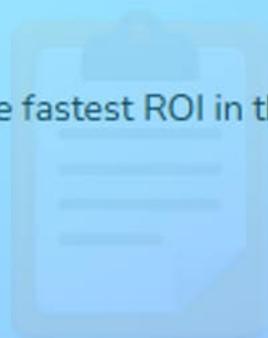
Progressive stack

Instructions to Skills to Prompts to Agents — each primitive builds on the last, adding capability without replacing what came...

PART 1

Instructions

Always-on foundation — the fastest ROI in the entire stack



PART 2

Skills

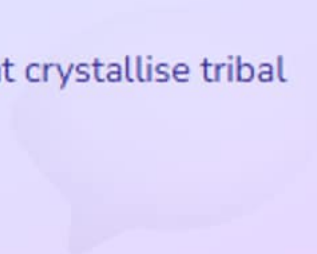
On-demand capability packs loaded by AI relevance matching



PART 3

Custom Prompts

Team slash commands that crystallise tribal knowledge



PART 4

Agents

Constrained personas composing all prior primitives



Click any section to jump directly there

Instructions

The 5-minute setup hook. One file. Every Copilot interaction inherits your team's conventions from this moment on.



Always-on baseline

copilot-instructions.md is injected into every request automatically — no activation step required.



Three surfaces

Repo-wide, path-scoped by applyTo glob, and AGENTS.md nearest-directory playbook — pick the right selector.



Transparent references

VS Code shows which instruction files were applied in the References panel — the system is verifiably working.

```
$ cat .github/copilot-instructions.md  
↳ One file. Every Copilot interaction inherits it.
```

Three Instruction Surfaces — One for Each Scope



Repo-wide

The whole-repository baseline
— always active.

`.github/copilot-instructions.md`

Applied to every request in this
repo

Coding standards, build
commands, file structure

No frontmatter needed — pure
Markdown



Path-scoped

Activates only for matching file
patterns.

`.github/instructions/*.instructions.md`

applyTo: glob pattern in
frontmatter

Language-specific rules, test
conventions

Combines with repo-wide when
both match



AGENTS.md

Nearest-directory agent
playbook — open format.

AGENTS.md at repo root or any
subdirectory

Commands, test steps, PR
expectations

Monorepo-safe: each subproject
gets its own

Cross-agent portable — not VS
Code only

A Production-Ready copilot-instructions.md

.github/copilot-instructions.md

Repository Instructions

This repository uses TypeScript with strict type checking.

Build and Test

- Build: `npm run build`
- Tests co-located in `\_\_tests\_\_` directories
- Use Vitest – never Mocha or Jest
- Run: `npm test`

Coding Standards

- Prefer functional patterns
- Explicit return types on all functions
- JSDoc on every exported function
- Named exports only – no default exports

Error Handling

- Custom error classes extending Error
- Structured logging via `logger.error()`
- Never swallow errors silently

⚡ Immediate effect

Created this file? Every Copilot request in this repo now inherits your conventions — no restart or re-configuration.

📄 Keep it under 2 pages

Bloated instruction files consume context budget meant for your actual question. Distill to true conventions only.

🔍 Verify via References

After the first chat response, check the References panel in VS Code — instruction files used are listed there.

Path-Scoped Instructions — Precision at File Level

```
.github/instructions/models.instructions.md
```

```
---  
applyTo: "src/models/**/*.ts"  
---
```

Database Model Instructions

When working with database models:

1. Use Prisma schema in `prisma/schema.prisma`
2. Include JSDoc with field descriptions
3. Define relationships with `@relation`
4. Add indexes for foreign keys
5. snake_case for DB columns, camelCase in TS
6. Always include: `createdAt` and `updatedAt`

Surgical scoping

applyTo: glob means these rules only activate when Copilot is working on matching files — never polluting other contexts.

Additive with repo-wide

Path-scoped instructions combine with copilot-instructions.md — both apply when a file matches both selectors.

Store in .github/instructions/

All .instructions.md files live in one place — easy to audit, review in PRs, and onboard new contributors.

AGENTS.md — Cross-Tool Institutional Knowledge

What goes in AGENTS.md

Setup and install commands for the repo

Test commands and linting steps for PRs

Repository navigation tips and subproject layout

Per-directory guardrails in monorepos

PR title format, commit conventions, workflow rules

Why it outlasts any single tool

Open Markdown format — not VS Code-specific

Any coding agent that reads AGENTS.md gets the playbook

Nested files: frontend/AGENTS.md +
infra/AGENTS.md

Subproject-scoped rules without a monolithic root file

Institutional knowledge that travels with the repo

Choosing the Right Instruction Surface

Always, whole
repo

copilot-instructions.md — the repo constitution, active on every request, zero conditions.

Repo-wide

Only these files

.instructions.md with applyTo glob — language-specific rules, test conventions, model patterns.

Path-scoped

This directory's
commands

AGENTS.md — setup, test, and PR rules scoped to a subproject or monorepo package.

AGENTS.md

Generated
baseline

Run /init in Copilot Chat — auto-generates copilot-instructions.md from your workspace.

Auto-generate

Verify it's
working

Check the References panel in every chat response — applied instruction files are listed there.

Debug tip

PART 2

Skills

GA as of v1.109 — the primitive that shifts Copilot from passive context to active capability.



AI relevance matching

Copilot loads the right skill without the developer doing anything — specialized expertise on demand.



Progressive loading

Discovery metadata loads always; full skill body and scripts only load when the task matches.



Portable open standard

Skills work in VS Code, Copilot CLI, and GitHub Copilot coding agent — write once, use everywhere.

```
$ # Skill loaded automatically when you ask to run tests  
↓ AI relevance matching – zero invocation friction
```

Three-Level Progressive Loading

1

Discovery

Lightweight metadata scanned at startup.

Skill name and description only

Always loaded — near-zero context cost

Powers AI relevance matching

Dozens of skills with minimal overhead

2

Instructions

Full SKILL.md body — when the task matches.

Complete guidance and process steps

Loaded only when description matches prompt

Context-efficient — not always present

Contains scripts, examples references

3

Resources

Scripts and templates — when referenced.

Shell scripts, test templates, examples

Fetches lazily during execution

Never loaded if not referenced

Keeps even complex skills lightweight

A Test-Runner Skill — Anatomy

`.github/skills/test-runner/SKILL.md`

```
---
name: test-runner
description: Run tests, analyze failures, and suggest
  fixes for unit and integration tests. Use when asked
  to test, debug failures, or add test coverage.
---
```

`# Test Runner Skill`

`## When to Use`

- User asks to test an API endpoint
- Debugging failing integration tests

`## Process`

1. Check existing tests in `tests/api/``
2. Use the template in `[template](./test-template.ts)`
3. Run: ``npm run test:api``
4. Analyze failures and suggest targeted fixes

Description drives matching

The description frontmatter is the matching key — Copilot compares your prompt semantically against every installed skill description.

Resource links are lazy

The `[template](./test-template.ts)` reference is only fetched if Copilot references it during execution — not on skill load.

Store in `.github/skills/`

Each skill gets its own subdirectory: `SKILL.md` + `scripts/` + `examples/`. Personal skills live in `~/copilot/skills/`.

Custom Prompts

Team slash commands — tribal knowledge crystallised into /commands any developer can invoke.



Anatomy first

.github/prompts/ path, frontmatter schema, /command binding, and variable interpolation.



Team sharing unlock

One developer writes the prompt file. Every developer on the team gets the slash command.



Instructions vs Prompts

Instructions load passively. Prompts are invoked explicitly — the developer decides when.

```
$ /component MyButtonGroup  
↳ Scaffolds a full React component with tests
```

Prompt File Anatomy

.github/prompts/component.prompt.md

```
---
name: component
description: Generate a React component with TypeScript,
  tests, and documentation
tools: ["editFiles", "createFile"]
agent: agent
model: Claude Sonnet 4 (copilot)
---
```

Component Generator

Create a new React component: {{componentName}}

Files to Create

```
src/components/{{componentName}}/
  {{componentName}}.tsx
  {{componentName}}.types.ts
  {{componentName}}.module.css
__tests__/{{componentName}}.test.tsx
index.ts
```

⚡ Tool and agent scoping

tools: restricts what the prompt can do. agent: agent gives it full agentic capability. model: lets you pick the best model per task.

abc Variable interpolation

{{componentName}} becomes a parameter. Also supports \${selection}, \${file}, and \${input:name:placeholder} for dynamic content.

🔗 Reference instructions

Link to copilot-instructions.md inside the prompt body — reuse conventions without duplicating them into every prompt file.

Skills vs. Custom Prompts — Know Which to Reach For

⚡ Skills — AI activates them

Loaded automatically when the task description matches

Developer does nothing — relevance matching fires

Best for: specialized capabilities and repeatable expertise

Example: test-runner loads when you ask to test an endpoint

Stored in `.github/skills/` — open standard across tools

💬 Prompts — Developer invokes them

Developer types `/component` or selects from the list

Explicit invocation — the command is the intent signal

Best for: scaffolding, checklists, and structured workflows

Example: `/component MyForm` creates the full file tree

Stored in `.github/prompts/` — visible to all team members

Agents

Synthesis peak — constrained personas composing all prior primitives. Tool boundaries are the architecture.



Constraint as design

A read-only Planner cannot accidentally edit files. The tool restriction is not a workaround — it is the architecture.



Planner-to-implementer handoff

After planning, one button transitions to an implementer agent with full context preserved — the whoa moment.



Decision tree payoff

The talk closes with a durable answer to which primitive do I reach for — earned after experiencing all four.

\$ # Select Planner agent → research → handoff → Implementer
↳ Tool constraints enforcing role separation by design

The Planner Agent — Constraint as Architecture

.github/agents/planner.agent.md

```
---
name: planner
description: Generate implementation plans by researching
  the codebase. Read-only – never modifies files.
tools: ["search", "readFile", "listFiles", "fetch"]
model: Claude Sonnet 4 (copilot)
handoffs:
  - label: Start Implementation
    agent: agent
    prompt: Implement the plan outlined above.
    send: false
---
```

Planning Agent

You are a senior architect creating implementation plans.

Rules

- NEVER modify files – you are read-only
- Always cite files and line numbers
- Include effort estimates for each step

tools: is the boundary

search, readFile, listFiles, fetch — no editFiles, no terminal. The Planner literally cannot write code. Intentional by design.

Handoff button appears

After planning, a Start Implementation button appears in chat. One click transitions to the implementer with full plan context.

Model per agent

Pick the best model for each role — a reasoning model for planning, a fast model for implementation. Not one-size-fits-all.

Advanced Agent Patterns



Planner → Implementer

Read-only Planner researches and plans; handoff to Implementer with full context — role separation enforced by tools.



Security reviewer

search + readFile only — the security agent scans for vulnerabilities without the ability to change code it reviews.



Database admin agent

terminal + SQL tools enabled; frontend directories excluded from listFiles — the right capability set, no more.



Model selection per agent

Planner uses a reasoning model for deep analysis; Implementer uses a fast model for code generation — match model to task.

The Configuration Stack — How the Primitives Compose

Instructions

Foundation — always-on baseline every other primitive inherits



Skills

Capabilities — on-demand expertise loaded by AI relevance matching



Prompts

Workflows — team slash commands encoding repeatable tasks



Agents

Personas — constrained roles composing instructions + skills + prompts



Choosing the Right Primitive

Passive context

Copilot should always know your repo conventions, standards, and build commands.

→ [Instructions](#)

Directory rules

A subproject or monorepo package needs its own commands, tests, and PR guidance.

→ [AGENTS.md](#)

Active capability

Specialized expertise should load automatically when the task matches — no invocation.

→ [Skills](#)

Repeatable workflow

A task is well-defined and the team should share a consistent /command for it.

→ [Prompts](#)

Full persona

A role needs specific tools enabled or disabled, and possibly a handoff to another agent.

→ [Agents](#)

From Repeated Context to Inherited Knowledge

Before

Developer re-explains conventions in every chat session — minutes wasted, every day

Copilot suggests raw SQL when the project uses Prisma — zero codebase awareness

Different developers get different response quality based on how much context they include

Generic boilerplate: no types, console.log errors, wrong test framework

After

copilot-instructions.md injected automatically — conventions always present, zero effort

Path-scoped instructions load Prisma patterns only for model files — surgical precision

Every team member gets the same configured baseline from day one — shared quality floor

TypeScript endpoint, custom error classes, Vitest tests, JSDoc — the first time

5 min

To create your first instructions file and see immediate improvement

0 effort

Per-request — instructions inject automatically once committed

4 primitives

Progressive stack from foundation to orchestration

✓ What You Can Do Today

Today

- ❑ Create `.github/copilot-instructions.md` with your build commands and coding standards
- ❑ Ask Copilot to generate a first draft using the `/init` command in chat
- ❑ Verify the References panel shows the file applied on your next request

This Week

- ❑ Add one path-scoped `.instructions.md` for your most convention-heavy file pattern
- ❑ Write one `.prompt.md` for a scaffolding workflow your team repeats — commit it to the repo
- ❑ Audit whether any team conventions are buried in wikis that should be in instructions files

This Month

- ❑ Create a `SKILL.md` for a specialized capability your team uses regularly
- ❑ Author a Planner agent with read-only tools — try the handoff to an implementer once
- ❑ Use the decision tree: for every new customization need, pick the right primitive first

🔑 Key Takeaway

Start with instructions. Add skills and agents only when a clear need emerges. Over-engineering primitives is real — resist it.

 VS Code Documentation[Customize AI in Visual Studio Code](#)

Overview of all customization options — the canonical starting point

[Use custom instructions in VS Code](#)

Instructions syntax, applyTo globs, and personal vs. workspace scoping

[Use Agent Skills in VS Code](#)

Skill structure, progressive loading, and the open standard

[Use prompt files in VS Code](#)

Prompt frontmatter schema, variable interpolation, and team sharing

[Custom agents in VS Code](#)

Agent file structure, tool restrictions, model selection, and handoffs

 GitHub Documentation[Adding repository custom instructions for GitHub Copilot](#)

GitHub-side documentation for repository instructions

[AGENTS.md open format](#)

Open specification for the cross-agent AGENTS.md playbook format



Copilot Configuration Primitives

Encode your codebase once. Every interaction inherits it.



Instructions

5 minutes — one file, immediate improvement, zero ongoing maintenance



Skills

Expertise on demand — AI relevance matching, zero invocation friction



Agents

Constrained personas — tool boundaries as architecture, not workarounds

Create `.github/copilot-instructions.md` today. Add skills and agents only when a clear need emerges.